

Universidade da Beira Interior

Departamento de Informática



**Departamento de
Informática**

Uma Abordagem Híbrida Quântico-Clássica ao Escalonamento de Processos via QAOA

Elaborado por:

Tomás Simões Duarte

Orientador:

Professor Doutor Paul Andrew Crocker

19 de junho de 2026

Agradecimentos

A conclusão deste trabalho, bem como da grande maior parte da minha vida académica não seria possível sem a ajuda de ...

Conteúdo

Conteúdo	ii
Lista de Figuras	iii
Lista de Tabelas	iv
Lista de Equações	v
1 Introdução	1
1.1 Formulação do Problema	1
2 Formulação QUBO para Escalonamento	2
2.1 CPU Scheduling em Sistemas Operativos	2
2.2 Formulação QUBO	2
2.3 Construção da Matriz Q	3
2.4 Exemplo Ilustrativo	4
3 Resolução do QUBO com QAOA	6
3.1 Conversão do QUBO em Hamiltoniano	6
3.2 Mixer X e XY	7
3.2.1 Mixer X	8
3.2.2 Mixer XY	8
3.3 Circuito Variacional e Otimização Clássica	9
3.4 Medição, Decodificação e Seleção da Solução	9
Bibliografia	11

Lista de Figuras

Lista de Tabelas

Lista de Equações

2.1	Forma geral de um problema QUBO	2
2.2	Desvio quadrático total das cargas	3
2.3	Carga média alvo	3
2.4	Energia QUBO de uma solução	3
2.5	Carga total no núcleo j	3
2.6	Expansão do termo de balanceamento	3
2.7	Coefficiente diagonal do balanceamento	3
2.8	Penalidade one-hot	4
2.9	Coefficientes da matriz Q	4
2.10	Indexação linear das variáveis no exemplo	4
2.11	Termos diagonais do balanceamento no exemplo	4
2.12	Matriz de balanceamento do exemplo	5
2.13	Matriz Q final do exemplo	5
2.14	Comparação de energia entre duas soluções viáveis	5
3.1	Energia QUBO a converter em Hamiltoniano	6
3.2	Mapeamento de variável binária para operador de Pauli Z	6
3.3	Conversão de um termo linear do QUBO	6
3.4	Conversão de um termo quadrático do QUBO	7
3.5	Hamiltoniano de custo	7
3.6	Unitária de custo do QAOA	7
3.7	Fase induzida pela energia da bitstring	7
3.8	Unitária de mistura do QAOA	7
3.9	Hamiltoniano do mixer X	8
3.10	Estado inicial uniforme do mixer X	8
3.11	Estados possíveis para um processo e dois núcleos	8
3.12	Estados one-hot válidos	8
3.13	Bloco de qubits associado a um processo	8
3.14	Interação do mixer XY	8
3.15	Troca de amplitude entre atribuições	9
3.16	Dimensão do espaço binário total	9
3.17	Número de atribuições one-hot válidas	9
3.18	Circuito QAOA com p camadas	9
3.19	Otimização dos parâmetros variacionais	9
3.20	Energia clássica usada na seleção final	10

Acrónimos

CFS *Completely Fair Scheduler*

CPU *Central Processing Unit*

GPU *Graphics Processing Unit*

NISQ *Noisy Intermediate-Scale Quantum*

QAOA *Quantum Approximate Optimization Algorithm*

QUBO *Quadratic Unconstrained Binary Optimization*

Introdução

Os sistemas operativos modernos distribuem carga entre núcleos de *Central Processing Unit* (CPU) recorrendo a heurísticas clássicas [1] que, apesar de eficazes em produção, tomam decisões subótimas em cenários de elevada contenção de recursos.

Em paralelo, métodos de otimização quântica, como o *Quantum Approximate Optimization Algorithm* (QAOA) [2], têm sido investigados em problemas combinatoriais que podem ser formulados como *Quadratic Unconstrained Binary Optimization* (QUBO) [3, 4].

No entanto, a era *Noisy Intermediate-Scale Quantum* (NISQ) impõe limitações de escala e ruído que tornam difícil avaliar quando um escalonador quântico oferece vantagens concretas sobre alternativas clássicas [5]. Preskill antecipa que esta barreira será ultrapassada com hardware da ordem do *megaquop* — máquinas capazes de executar 10^6 operações quânticas com erro tolerável [6] — fornecendo um referencial concreto para situar a relevância futura deste tipo de abordagem.

Este relatório apresenta uma plataforma híbrida quântico-clássica para testar o problema de atribuição de processos a núcleos de CPU. A plataforma não substitui o escalonador do kernel; isola a decisão de atribuição processo-núcleo, modela-a como um QUBO, resolve-a com QAOA via PennyLane [7], e valida os resultados por comparação com um *solver* exato de *brute force* quando o tamanho da instância o permite. O objetivo não é demonstrar superioridade quântica, mas caracterizar o comportamento do QAOA neste domínio em função da escala do problema.

1.1 Formulação do Problema

Dado um conjunto de processos com utilização de CPU conhecida e um número fixo de núcleos, o sistema deve atribuir cada processo a exatamente um núcleo. O objetivo de otimização é o equilíbrio de carga: a carga total de cada núcleo deve aproximar-se o mais possível da carga média global. A solução deve ainda respeitar restrições de atribuição exclusiva — nenhum processo pode ficar por atribuir ou ser atribuído a múltiplos núcleos simultaneamente.

Expressar estas restrições e este objetivo numa formulação QUBO, compatível com otimizadores quânticos, constitui o problema central deste projeto.

Formulação QUBO para Escalonamento

2.1 CPU Scheduling em Sistemas Operativos

Um escalonador de CPU determina que processo ou *thread* executa em cada núcleo do processador em cada instante. Escalonadores como o *Completely Fair Scheduler* (CFS) do Linux consideram simultaneamente latência, *throughput*, prioridade, afinidade de núcleo e localização de cache, tomando decisões em microssegundos com base em heurísticas clássicas [1].

No entanto, a decisão de atribuição de processos a núcleos é fundamentalmente um problema de partição combinatorial. Dado um conjunto de N processos com cargas w_i e K núcleos, o número de atribuições possíveis cresce como K^N , tornando a procura exaustiva para instâncias reais [3, 4].

Este projeto adota uma formulação simplificada deste problema: cada *workload entity* possui uma utilização de CPU conhecida, e o objetivo é distribuí-las pelos núcleos disponíveis minimizando o desequilíbrio de carga. Esta abstração isola a componente combinatorial do escalonamento sem replicar a complexidade de um escalonador de kernel completo.

2.2 Formulação QUBO

Um problema QUBO [3, 4] consiste em minimizar uma expressão quadrática sobre variáveis binárias $x_i \in \{0, 1\}$, representada de forma compacta como:

$$\min_{\mathbf{x} \in \{0,1\}^n} \mathbf{x}^\top \mathbf{Q} \mathbf{x} = \sum_i \sum_j Q_{ij} \cdot x_i \cdot x_j \quad (2.1)$$

onde $\mathbf{Q} \in \mathbb{R}^{n \times n}$ é uma matriz cujos coeficientes codificam o problema a resolver. Expandindo o somatório, surgem dois tipos de termos:

- **Termos diagonais** ($i = j$): como $x_i^2 = x_i$ para variáveis binárias, $Q_{ii} \cdot x_i \cdot x_i = Q_{ii} \cdot x_i$. Estes termos associam um custo individual a cada variável — um coeficiente $Q_{ii} < 0$ incentiva o otimizador quântico a escolher $x_i = 1$; um coeficiente $Q_{ii} > 0$ incentiva $x_i = 0$.
- **Termos fora da diagonal** ($i \neq j$): $Q_{ij} \cdot x_i \cdot x_j$ contribui para a energia apenas quando $x_i = x_j = 1$. Um coeficiente positivo penaliza a ativação simultânea de x_i e x_j ; um coeficiente negativo recompensa-a.

A energia de uma solução $\mathbf{x}$ é o valor $\mathbf{x}^\top Q \mathbf{x}$. A solução ótima $\mathbf{x}^*$ é a atribuição binária que minimiza esta energia no espaço de 2^n configurações possíveis.

2.3 Construção da Matriz Q

Dado um conjunto de N processos com cargas w_i e K núcleos, define-se uma variável binária $x_{ij} \in \{0, 1\}$ para cada par processo-núcleo. Esta variável toma o valor 1 quando o processo i é atribuído ao núcleo j , e o valor 0 caso contrário. Na implementação, a variável x_{ij} é armazenada no índice linear $iK + j$ da matriz Q .

O objetivo original do problema é equilibrar a carga entre núcleos. Para isso, mede-se o desequilíbrio através do desvio quadrático total das cargas relativamente à carga média:

$$\min \sum_{j=0}^{K-1} (L_j - L_{avg})^2. \quad (2.2)$$

onde

$$L_{avg} = \frac{W_{total}}{K}. \quad (2.3)$$

Para obter a formulação QUBO, este termo é expandido e combinado com penalidades que impõem atribuições válidas. O resultado é codificado na matriz Q , de modo que o *solver* minimize a energia

$$E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x}. \quad (2.4)$$

Assim, minimizar a energia QUBO corresponde, por construção, a procurar uma atribuição simultaneamente equilibrada e válida.

Para cada núcleo j , a carga total é dada por:

$$L_j = \phi_j + \sum_{i=0}^{N-1} w_i x_{ij}, \quad (2.5)$$

onde ϕ_j representa carga já comprometida nesse núcleo, relevante para a secção da decomposição iterativa. Na primeira iteração, ou num QUBO sem carga fixa, tem-se $\phi_j = 0$.

Substituindo L_j no termo de um núcleo e usando $x_{ij}^2 = x_{ij}$, obtém-se:

$$\begin{aligned} (L_j - L_{avg})^2 &= \left(\phi_j + \sum_{i=0}^{N-1} w_i x_{ij} - L_{avg} \right)^2 \\ &= \underbrace{\sum_{i=0}^{N-1} d_{ij} x_{ij}}_{\text{termos diagonais}} + 2 \underbrace{\sum_{0 \leq i < k} w_i w_k x_{ij} x_{kj}}_{\text{termos de co-localização}} + \underbrace{(\phi_j - L_{avg})^2}_{\text{constante}}, \end{aligned} \quad (2.6)$$

onde

$$d_{ij} = w_i^2 + 2w_i \phi_j - 2L_{avg} w_i. \quad (2.7)$$

Estes coeficientes são então adicionados à matriz Q . Os termos diagonais estão associados à ativação individual de uma variável x_{ij} ; os termos de co-localização aparecem quando dois processos distintos são atribuídos ao mesmo núcleo.

Além do objetivo de balanceamento, é necessário garantir que cada processo é atribuído a exatamente um núcleo. Para isso adiciona-se a penalidade one-hot:

$$P \sum_{i=0}^{N-1} \left(\sum_{j=0}^{K-1} x_{ij} - 1 \right)^2. \quad (2.8)$$

Esta penalidade segue a prática comum de converter restrições de igualdade em termos quadráticos penalizados em QUBO [4]. Neste caso, contribui com $-P$ na diagonal de cada variável e com $+P$ entre variáveis do mesmo processo em núcleos diferentes.

Combinando o objetivo de balanceamento e a penalidade one-hot, cada entrada da matriz é determinada pelo tipo de relação entre as duas variáveis consideradas:

$$Q[x_{ij}, x_{kl}] = \begin{cases} w_i^2 + 2w_i\phi_j - 2L_{avg}w_i - P, & \text{se } i = k \text{ e } j = l, \\ w_i w_k, & \text{se } i \neq k \text{ e } j = l, \\ P, & \text{se } i = k \text{ e } j \neq l, \\ 0, & \text{se } i \neq k \text{ e } j \neq l. \end{cases} \quad (2.9)$$

A primeira linha corresponde à contribuição individual de cada variável. A segunda linha penaliza processos distintos que partilham o mesmo núcleo, enquanto a terceira corresponde à penalidade one-hot entre núcleos diferentes do mesmo processo. A última linha indica que variáveis de processos e núcleos diferentes não interagem diretamente.

2.4 Exemplo Ilustrativo

Considere-se $N = 2$ processos e $K = 2$ núcleos, com cargas $w_0 = 0.3$ e $w_1 = 0.5$, sem carga fixa ($\phi_j = 0$). As variáveis ficam ordenadas como:

índice	0	1	2	3
variável	x_{00}	x_{01}	x_{10}	x_{11}

(2.10)

A carga média é $L_{avg} = (0.3 + 0.5)/2 = 0.4$.

Antes de adicionar a penalidade one-hot, os termos diagonais do balanceamento são:

$$\begin{aligned} Q_{0,0}^{bal} = Q_{1,1}^{bal} &= w_0^2 - 2L_{avg}w_0 = 0.09 - 2(0.4)(0.3) = -0.15, \\ Q_{2,2}^{bal} = Q_{3,3}^{bal} &= w_1^2 - 2L_{avg}w_1 = 0.25 - 2(0.4)(0.5) = -0.15. \end{aligned} \quad (2.11)$$

Para o núcleo 0, a expansão de $(L_0 - L_{avg})^2$ contém o termo de co-localização:

$$2w_0w_1x_{00}x_{10} = 0.30x_{00}x_{10}.$$

De forma análoga, a expansão de $(L_1 - L_{avg})^2$ contém:

$$2w_0w_1x_{01}x_{11} = 0.30x_{01}x_{11}.$$

Como Q é simétrica, este contributo é dividido pelas duas posições simétricas da matriz.

Assim, a matriz do objetivo de balanceamento é:

$$Q_{bal} = \begin{pmatrix} -0.15 & 0 & 0.15 & 0 \\ 0 & -0.15 & 0 & 0.15 \\ 0.15 & 0 & -0.15 & 0 \\ 0 & 0.15 & 0 & -0.15 \end{pmatrix}. \quad (2.12)$$

Com $P = 1.0$, a penalidade one-hot subtrai 1.0 a cada termo diagonal e adiciona 1.0 entre as variáveis do mesmo processo em núcleos distintos: x_{00} com x_{01} , e x_{10} com x_{11} . A matriz final fica:

$$Q = \begin{pmatrix} -1.15 & 1.0 & 0.15 & 0 \\ 1.0 & -1.15 & 0 & 0.15 \\ 0.15 & 0 & -1.15 & 1.0 \\ 0 & 0.15 & 1.0 & -1.15 \end{pmatrix}. \quad (2.13)$$

Para verificar o efeito dos termos de co-localização, comparem-se duas soluções viáveis: A, com os processos distribuídos ($x_{00} = x_{11} = 1$), e B, com ambos os processos no núcleo 0 ($x_{00} = x_{10} = 1$). Usando $E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x}$:

$$\begin{aligned} E_A &= Q_{0,0} + Q_{3,3} = -1.15 + (-1.15) = -\mathbf{2.30}, \\ E_B &= Q_{0,0} + Q_{2,2} + Q_{0,2} + Q_{2,0} = -1.15 + (-1.15) + 0.15 + 0.15 = -\mathbf{2.00}. \end{aligned} \quad (2.14)$$

Como $E_A < E_B$, a solução distribuída tem menor energia. A matriz penaliza, portanto, a concentração dos dois processos no mesmo núcleo, favorecendo uma distribuição mais equilibrada.

Depois de construída a matriz Q , o problema de escalonamento passa a estar expresso numa forma compatível com otimizadores QUBO. O próximo passo será resolver esta matriz, isto é, procurar a bitstring $\mathbf{x}$ que minimiza a energia $E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x}$. Neste trabalho, essa etapa é realizada com QAOA [2], implementado em PennyLane [7].

Resolução do QUBO com QAOA

O QAOA é um algoritmo variacional híbrido para otimização combinatória, no qual um circuito quântico é combinado com um otimizador clássico para minimizar o valor esperado de um Hamiltoniano de custo [2, 8]. Neste projeto, a implementação do circuito e da otimização é feita com PennyLane [7].

Importa realçar que a execução foi realizada em simulação, e não em hardware quântico real. Esta opção permite evitar a latência associada aos dispositivos quânticos remotos, bem como os efeitos de ruído físico. Na implementação, o solver tenta primeiro o backend `lightning.gpu` do PennyLane, quando há suporte para GPU. Quando este backend não está disponível, recorre a `lightning.qubit` como alternativa [9].

A abordagem implementada atribui um qubit lógico a cada par processo-núcleo. Assim, uma instância com N processos e K núcleos requer $N \cdot K$ qubits.

3.1 Conversão do QUBO em Hamiltoniano

O QAOA trabalha com um Hamiltoniano de custo H_C e não diretamente com a matriz Q . Assim, a energia QUBO

$$E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x} \quad (3.1)$$

tem de ser convertida em forma de operadores quânticos. Esta conversão segue a relação usual entre variáveis binárias e variáveis de *spin* usada em formulações Ising de problemas combinatórios [10]. Para isso, cada variável binária $x_i \in \{0, 1\}$ é mapeada para operadores de Pauli Z_i usando a substituição:

$$x_i = \frac{I - Z_i}{2}. \quad (3.2)$$

Esta substituição permite representar os valores binários 0 e 1 através dos dois estados da base computacional, $|0\rangle$ e $|1\rangle$. Quando o qubit está no estado $|0\rangle$, o operador Z_i tem valor $+1$, resultando em $x_i = 0$; quando está no estado $|1\rangle$, o operador Z_i tem valor -1 , resultando em $x_i = 1$.

Assim, um termo linear da diagonal do QUBO transforma-se em:

$$Q_{ii} x_i = Q_{ii} \frac{I - Z_i}{2} = \frac{Q_{ii}}{2} I - \frac{Q_{ii}}{2} Z_i. \quad (3.3)$$

Para os termos quadráticos, considerando $q_{ij} = Q_{ij} + Q_{ji}$, obtém-se:

$$q_{ij}x_i x_j = q_{ij} \frac{I - Z_i}{2} \frac{I - Z_j}{2} = \frac{q_{ij}}{4} (I - Z_i - Z_j + Z_i Z_j). \quad (3.4)$$

Desta forma, o Hamiltoniano de custo pode ser escrito como:

$$H_C = cI + \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j, \quad (3.5)$$

onde c é uma constante, h_i são os coeficientes lineares associados a cada qubit, e J_{ij} representa a interação entre pares de qubits.

Uma vez obtido o Hamiltoniano de custo H_C , este passa a definir a energia associada a cada estado. No QAOA, esta informação é usada através de uma operação unitária de custo [2]:

$$U_C(\gamma) = e^{-i\gamma H_C}, \quad (3.6)$$

onde γ é um parâmetro variacional. Esta operação pode ser vista como uma evolução temporal sob o Hamiltoniano H_C durante um tempo controlado por γ . Como H_C é diagonal na base computacional, cada bitstring recebe uma fase dependente da sua energia:

$$U_C(\gamma)|x\rangle = e^{-i\gamma E(x)}|x\rangle. \quad (3.7)$$

Logo, a camada de custo não altera diretamente a probabilidade de cada bitstring; altera as suas fases relativas. Essas fases tornam-se úteis quando é aplicada a camada de mistura, responsável por distribuir amplitude entre soluções.

A camada de mistura é definida por outro Hamiltoniano H_M e pelo operador unitário

$$U_M(\beta) = e^{-i\beta H_M}, \quad (3.8)$$

onde β é também um parâmetro variacional. Enquanto U_C traduz informação sobre a qualidade das soluções através das fases, U_M permite explorar novas soluções ao misturar amplitudes entre estados da base computacional.

3.2 Mixer X e XY

O Hamiltoniano H_M presente em (3.8) é designado por *mixer*. No QAOA, o *mixer* é responsável por distribuir amplitude entre estados da base computacional [2, 11]. Para este trabalho foram considerados dois tipos: X e XY.

A diferença entre eles está no espaço de soluções explorado. O *mixer X* atua sobre todos os qubits individualmente, permitindo explorar todo o espaço binário. Já o *mixer XY* é adaptado à restrição one-hot, pois atua apenas entre qubits que representam núcleos alternativos para o mesmo processo [12, 13].

3.2.1 Mixer X

O *mixer X* é o padrão da formulação do QAOA [2]. O seu Hamiltoniano de forma simplificada é:

$$H_M^X = \sum_i X_i \quad (3.9)$$

onde X_i é o operador de Pauli X aplicado ao qubit i .

Este mixer é aplicado numa superposição de estados uniforme, aplicando *Hadamard gates* no estado inicial.

$$|s\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle, \quad (3.10)$$

onde $n = N \cdot K$ é o número total de qubits. Assim, o circuito começa numa superposição sobre todos os bitstrings possíveis.

No entanto, para o problema do escalonamento este espaço contém estados inválidos. Por exemplo, para um único processo e dois núcleos, existem quatro estados possíveis:

$$|00\rangle, |01\rangle, |10\rangle, |11\rangle. \quad (3.11)$$

Assumindo que cada posição representa a atribuição do processo a um núcleo, apenas dois destes estados são válidos:

$$|10\rangle, |01\rangle. \quad (3.12)$$

Por isso, com o mixer X , a validade das soluções depende da penalidade one-hot incluída no QUBO.

3.2.2 Mixer XY

O *mixer XY* é mais adequado à estrutura deste problema, porque atua apenas entre os qubits que representam os possíveis núcleos de um mesmo processo. Para cada processo i , existe um bloco de K qubits:

$$x_{i0}, x_{i1}, \dots, x_{i,K-1}. \quad (3.13)$$

O mixer XY aplica operações entre pares de qubits dentro desse bloco, que têm a forma:

$$X_{ij}X_{ik} + Y_{ij}Y_{ik} \quad (3.14)$$

onde j e k representam dois núcleos distintos associados ao mesmo processo i . Esta operação permite transferir amplitude entre atribuições do mesmo processo, por exemplo:

$$|10\rangle \leftrightarrow |01\rangle. \quad (3.15)$$

Esta operação preserva o número de bits ativos no bloco. Assim, partindo de um estado válido, como $|10\rangle$ ou $|01\rangle$, o mixer XY não cria estados inválidos como $|00\rangle$ ou $|11\rangle$ [12, 13].

Na implementação, quando se usa o *mixer* XY , o estado inicial não é a superposição uniforme sobre todos os bitstrings. Em vez disso, é preparado um estado válido em que todos os processos começam atribuídos ao núcleo 0.

Deste modo, embora o circuito continue a ter $N \cdot K$ qubits e o espaço binário total tenha dimensão

$$2^{N \cdot K}, \quad (3.16)$$

o mixer XY restringe o subespaço one-hot, que contém

$$K^N \quad (3.17)$$

atribuições válidas. Por esta razão, este *mixer* é particularmente adequado para o problema de atribuição de processos a núcleos.

3.3 Circuito Variacional e Otimização Clássica

Depois de definidas as camadas de custo e de mistura, o circuito QAOA é construído alternando estas duas operações durante p camadas [2]:

$$|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle = U_M(\beta_p) U_C(\gamma_p) \cdots U_M(\beta_1) U_C(\gamma_1) |\psi_0\rangle. \quad (3.18)$$

Os vetores $\boldsymbol{\gamma} = (\gamma_1, \dots, \gamma_p)$ e $\boldsymbol{\beta} = (\beta_1, \dots, \beta_p)$ são parâmetros variacionais. O seu valor não é fixado manualmente; é ajustado por um otimizador clássico de forma a minimizar o valor esperado do Hamiltoniano de custo:

$$\min_{\boldsymbol{\gamma}, \boldsymbol{\beta}} \langle \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) | H_C | \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) \rangle. \quad (3.19)$$

A otimização é feita com o otimizador Adam do PennyLane [14], durante um número configurável de iterações. Em cada iteração, o circuito é avaliado, o valor esperado de H_C é calculado, e os parâmetros são atualizados para reduzir essa energia esperada.

3.4 Medição, Decodificação e Seleção da Solução

Após a otimização dos parâmetros variacionais, o circuito QAOA final define uma distribuição de probabilidades sobre os bitstrings possíveis.

A validade de cada bitstring é verificada impondo a restrição one-hot: cada processo deve ter exatamente uma variável ativa entre os seus K núcleos possíveis. Se um

processo não tiver nenhuma variável ativa, ou se tiver mais do que uma, o bitstring é considerado inválido.

Para cada bitstring viável, a energia clássica é recalculada diretamente a partir da matriz QUBO:

$$E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x}. \quad (3.20)$$

A solução final devolvida pelo solver corresponde ao bitstring viável com menor energia QUBO encontrado após a avaliação da distribuição final. Desta forma, a etapa quântica gera uma distribuição candidata, enquanto a etapa clássica final garante que a solução é uma atribuição válida e energeticamente favorável.

Bibliografia

- [1] The Linux Kernel Development Community. CFS scheduler. <https://docs.kernel.org/scheduler/sched-design-CFS.html>. Accessed: 2026-06-18.
- [2] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm, 2014.
- [3] Gary Kochenberger, Jin-Kao Hao, Fred Glover, Mark Lewis, Zhipeng Lü, Haibo Wang, and Yang Wang. The unconstrained binary quadratic programming problem: A survey. *Journal of Combinatorial Optimization*, 28(1):58–81, 2014.
- [4] Fred Glover, Gary Kochenberger, and Yu Du. A tutorial on formulating and using QUBO models, 2018.
- [5] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.
- [6] John Preskill. Beyond NISQ: The megaguop machine, 2025.
- [7] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, Shahnawaz Ahmed, Vishnu Ajith, M. Sohaib Alam, Guillermo Alonso-Linaje, B. AkashNarayanan, Ali Asadi, et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations, 2018.
- [8] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3:625–644, 2021.
- [9] Ali Asadi et al. Hybrid quantum programming with PennyLane Lightning on HPC platforms, 2024.
- [10] Andrew Lucas. Ising formulations of many NP problems. *Frontiers in Physics*, 2:5, 2014.
- [11] Stuart Hadfield, Zhihui Wang, Bryan O’Gorman, Eleanor G. Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz, 2017.
- [12] Zhihui Wang, Nicholas C. Rubin, Jason M. Dominy, and Eleanor G. Rieffel. XY-mixers: Analytical and numerical results for QAOA, 2019.
- [13] Franz G. Fuchs, Kjetil Olsen Lye, Halvor Møll Nilsen, Alexander J. Stasik, and Giorgio Sartor. Constrained mixers for the quantum approximate optimization algorithm, 2022.
- [14] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2014.